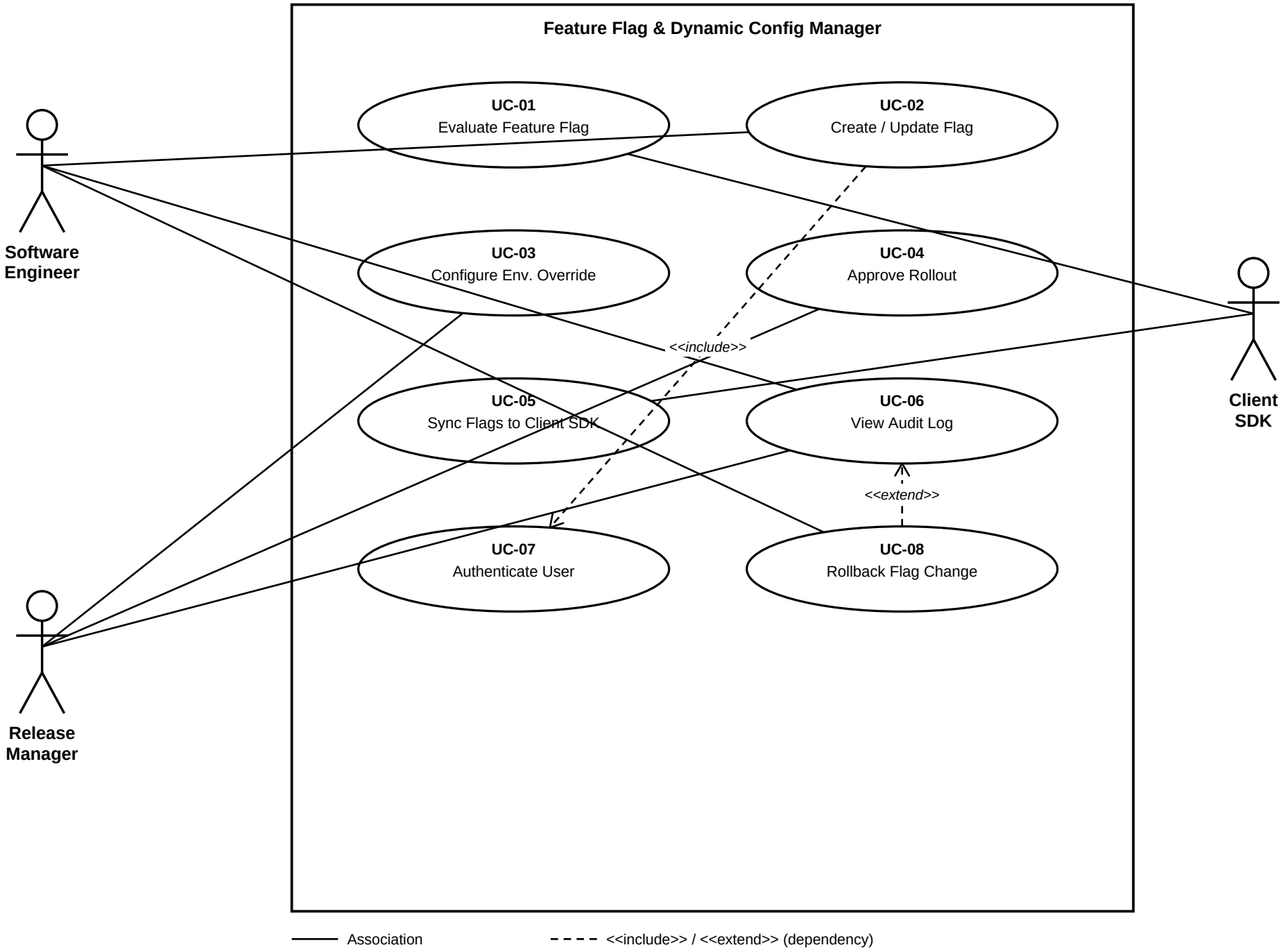
Requirements Table

Problem Statement #43 — Feature Flag & Dynamic Config Manager

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Functional	The system shall evaluate user targeting rules (e.g., User ID hash, beta cohort membership) and return an accurate boolean feature flag state to the requesting client.	High	Pass: A user in the target 10% rollout cohort receives the enabled flag. Fail: A user outside the cohort receives the enabled flag.	Correct, deterministic targeting is the core value proposition of the platform and underlies UC-01 (Evaluate Feature Flag).
FR-002	Functional	The system shall allow a Software Engineer to create and update a feature flag, including its rollout percentage and targeting rules, after authenticating.	High	Pass: A saved flag change is retrievable via the API/UI with the exact percentage and rules entered. Fail: A change is accepted without a valid authenticated session.	Supports UC-02 (Create/Update Flag), which <<include>>s UC-07 (Authenticate User) so every write is attributable to a real user.
FR-003	Functional	The system shall allow a Release Manager to configure environment-specific overrides (Dev, Staging, Prod) that take precedence over the default flag rule during evaluation.	Medium	Pass: Setting a Prod override to 'disabled' returns disabled for Prod clients even while Dev/Staging remain enabled. Fail: An override in one environment leaks into another.	Environment isolation prevents an in-progress experiment in one environment from unintentionally affecting production users.
FR-004	Functional	The system shall propagate flag and configuration changes to all connected client SDKs in real time without requiring a client restart or redeploy.	High	Pass: A flag toggled by an engineer is reflected in a connected SDK's in-memory state within 2 seconds. Fail: A connected SDK still serves the stale value after 2 seconds.	Realizes UC-05 (Sync Flags to Client SDK); near-instant propagation is what lets teams do safe, fast rollbacks.
FR-005	Functional	The system shall record an audit log entry (who, what changed, old value, new value, timestamp) for every flag or override change, and allow an authorized user to roll back a change to its previous value.	Medium	Pass: After a rollback action, the flag's value matches the pre-change value and a new audit entry records the rollback. Fail: A rollback silently fails or the audit trail is missing an entry.	Supports UC-06 (View Audit Log) and UC-08 (Rollback Flag Change, which <<extend>>s View Audit Log); required for incident response and compliance traceability.
NFR-001	Nonfunctional	The feature flag evaluation API must execute in under 15 ms (p99) to minimize overhead when integrated into client critical paths, and must enforce authenticated/authorized access to all write endpoints.	High	Pass: Benchmarking tests confirm p99 latency < 15 ms and security scans show no unauthenticated write access under simulated peak load.	Flag evaluation sits on the hot path of every feature-gated request, so latency directly impacts end-user experience; security prevents unauthorized cohort manipulation.
NFR-002	Nonfunctional	The system shall maintain 99.95% monthly availability for flag evaluation, and shall fail open by serving the last known-good cached configuration if the config service is unreachable.	High	Pass: During a simulated config-service outage, connected SDKs continue evaluating flags using cached values with zero client-facing errors. Fail: Clients throw errors or block requests when the service is unreachable.	Feature flagging infrastructure must never become a single point of failure for the applications that depend on it.

Use-Case Diagram: Feature Flag & Dynamic Config Manager



Use-Case Flow Specification

UC-02: Create / Update Feature Flag & Rollout — Feature Flag & Dynamic Config Manager

Use Case ID	UC-02
Use Case Name	Create / Update Feature Flag & Rollout
Primary Actor	Software Engineer
Secondary Actors	Release Manager (notified of change); Client SDK (receives synced flag)
Includes	UC-07: Authenticate User «include» — every create/update requires a verified, authenticated session.
Extends	UC-08: Rollback Flag Change «extend» — optionally triggered from this use case's history if a bad change needs reverting.
Preconditions	1. The Software Engineer has a valid account with permission to modify flags. 2. The target application/service is already registered in the platform. 3. The Config Manager service and its datastore are reachable.
Postconditions	Success: The flag's configuration (rollout %, targeting rules, environment overrides) is persisted, an audit log entry is created, and connected client SDKs receive the update within 2 seconds. Failure: No partial update is persisted; the prior flag state remains active and unchanged.

Main Success Scenario

1. Software Engineer selects “Create/Update Flag” from the dashboard.
2. System invokes UC-07 (Authenticate User) to verify the engineer's session and permissions.
3. System displays the flag editor form (flag key, description, targeting rules, rollout percentage, environment scope).
4. Engineer enters or edits the flag key, rollout percentage, and targeting rules, then submits.
5. System validates the input (e.g., rollout percentage between 0-100, no duplicate flag key).
6. System persists the flag configuration to the datastore and writes an audit log entry (who, old value, new value, timestamp).
7. System pushes the updated configuration to all connected client SDKs in real time.
8. System displays a confirmation message with the new flag state to the Engineer.
9. Use case ends successfully.

Alternate Flow — 5a. Validation Failure

5a. System detects an invalid input (e.g., rollout percentage out of range, or duplicate flag key).
5b. System displays a field-specific validation error and highlights the offending input.
5c. Engineer corrects the input and resubmits (flow returns to Main Success Scenario step 5), or cancels the operation, leaving the previous flag configuration unchanged.